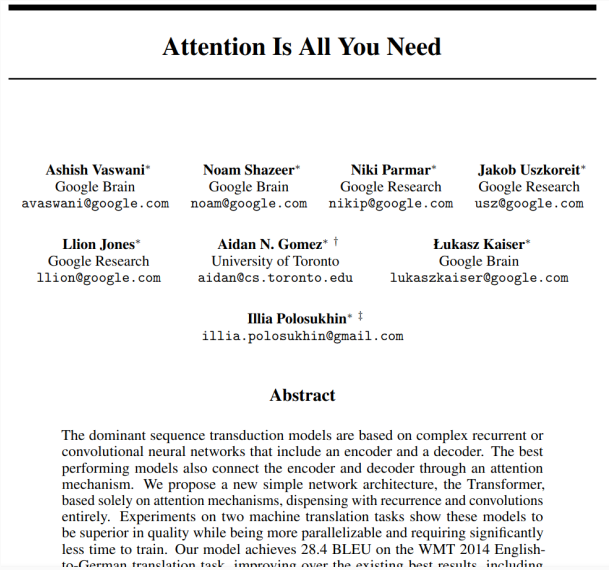
Natural Language Processing
10. Transformers

Juan José Alegría

May 27, 2025

Transformers

- Introduced in the seminal paper *Attention is All You Need* by researchers at Google [Vaswani et al., 2017].
- Originally designed to address the sequence-to-sequence (seq2seq) problem.
- Like RNN-based models, it consists of both an encoder and a decoder.
- Key innovation: replaces recurrence entirely with *attention* mechanisms — within the encoder, within the decoder, and between them.
    - In contrast, traditional RNN-based models use attention only between the encoder and decoder.



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including

What's Wrong with RNNs?

- RNNs process tokens sequentially, which limits parallelization: each token must wait for the previous one to be processed.
- Even with LSTMs or GRUs, long-range dependencies are difficult to capture — the path between any two tokens is $O(n)$ in length.
- Transformers address both issues:
    - All tokens are processed simultaneously, enabling efficient parallelization.
    - The path length between any two tokens is reduced to $O(1)$, improving the ability to model long-range dependencies.

Disclaimer

The whole lesson is based on the paper ‘Àttention is All You Need”

[Vaswani et al., 2017] and the amazing guide *The Illustrated Transformer*, by Jay Alammar (available here).

Architecture: (very) high-level look

Figure 1: A black box that maps an input sequence to an output sequence.

Architecture: zoom in +

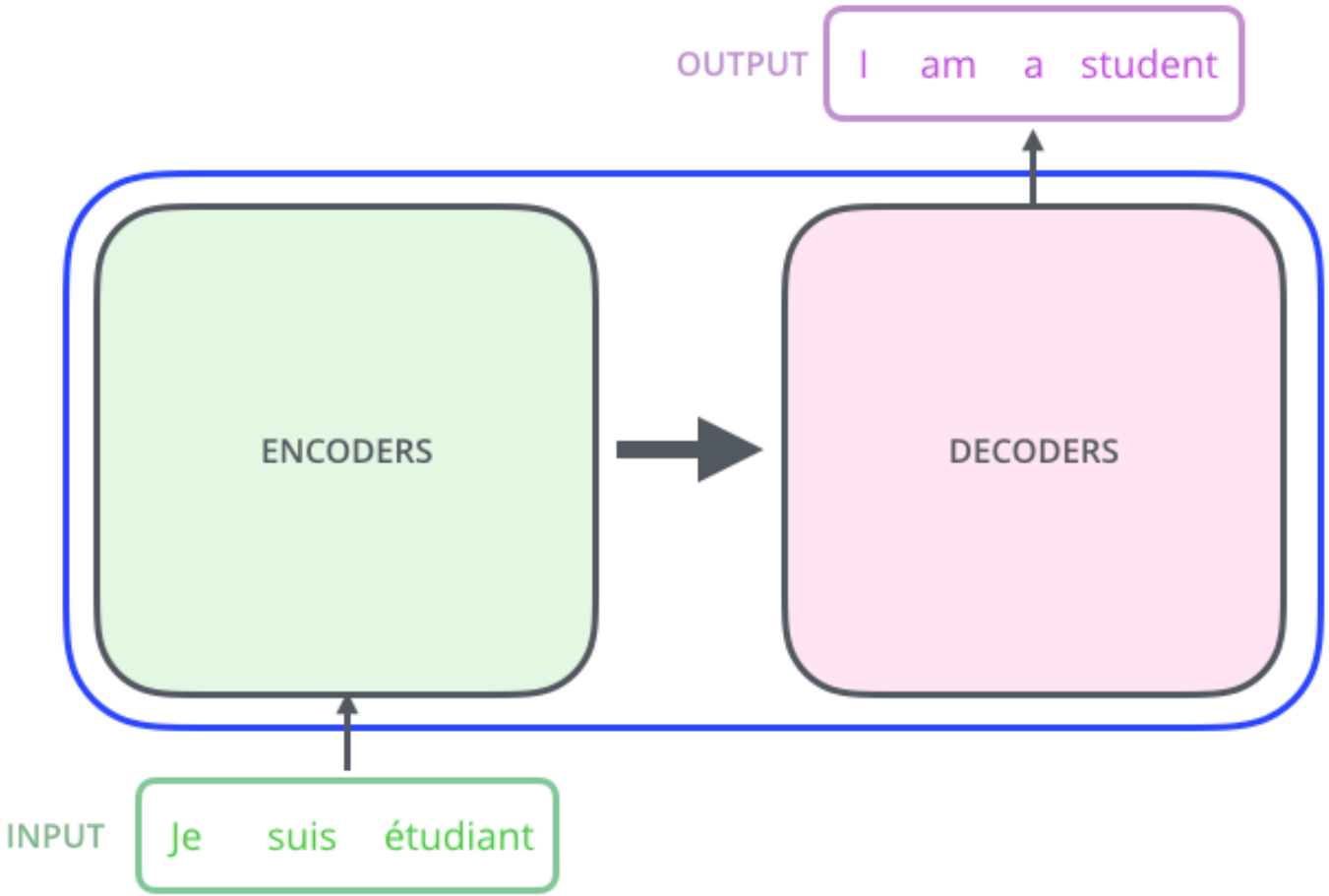


Figure 2: Two big blocks: an encoding component and a decoding component.

Architecture: zoom in ++

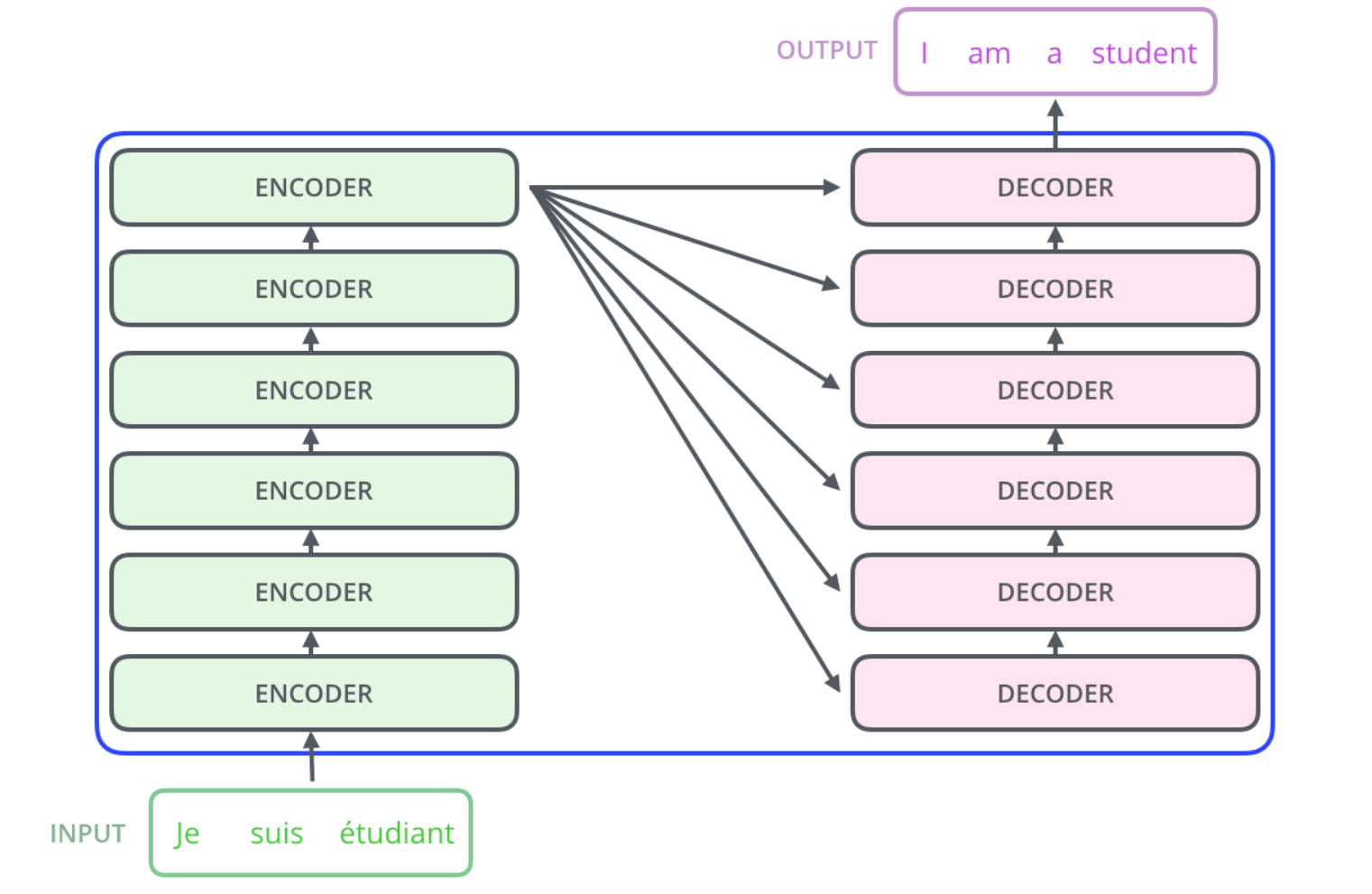


Figure 3: The encoding component is a stack of encoders, and the decoding component is a stack of decoders. In the original paper, $N = 6$ for both the encoding and decoding component.

Architecture: a single encoder

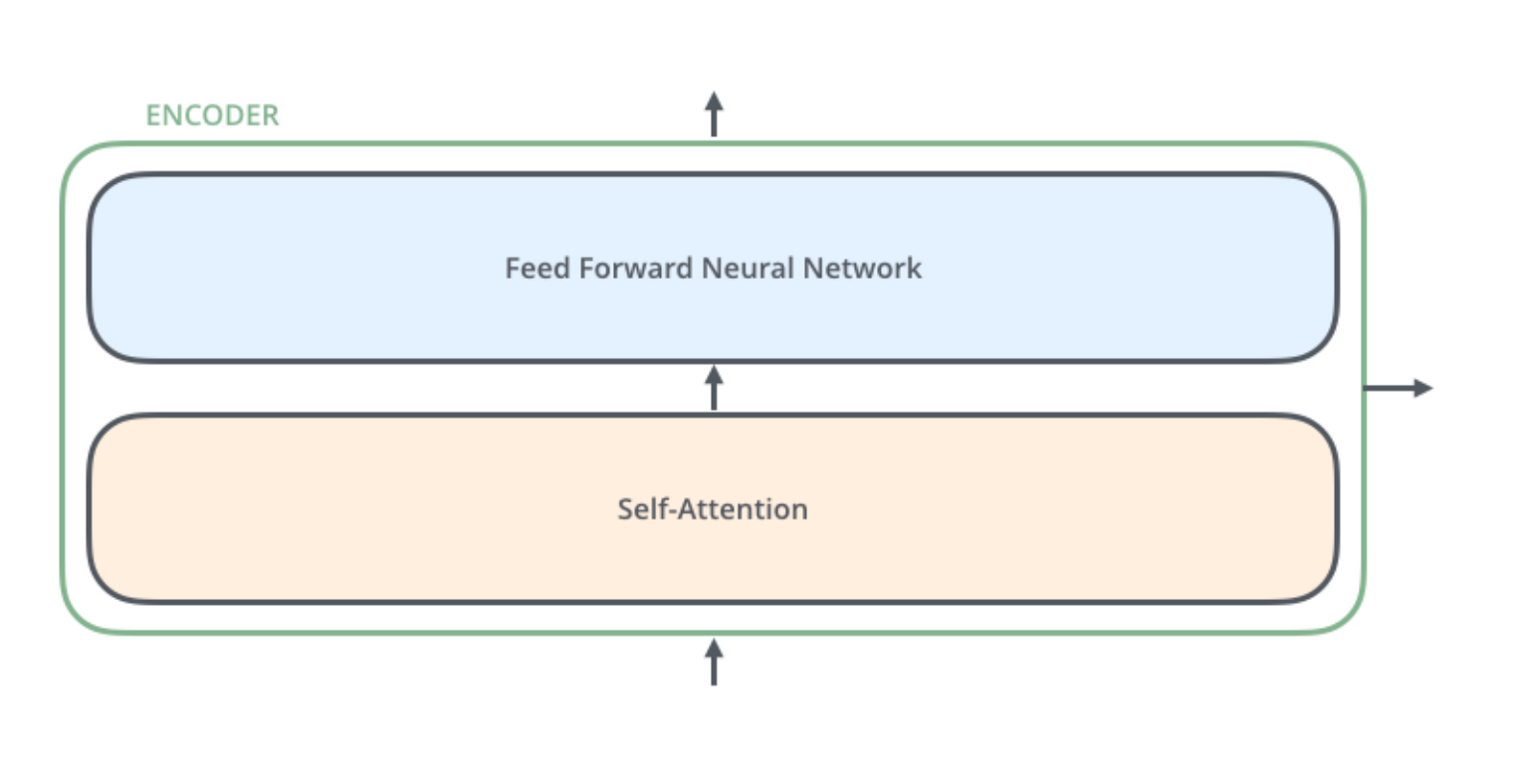


Figure 4: All the encoders have the same structure (but they don't share weights!). Just two layers: self-attention and a feed forward network.

Self-attention? A mechanism to look into other words of the input as it encodes an specific word (more on that soon).

Architecture: a single decoder

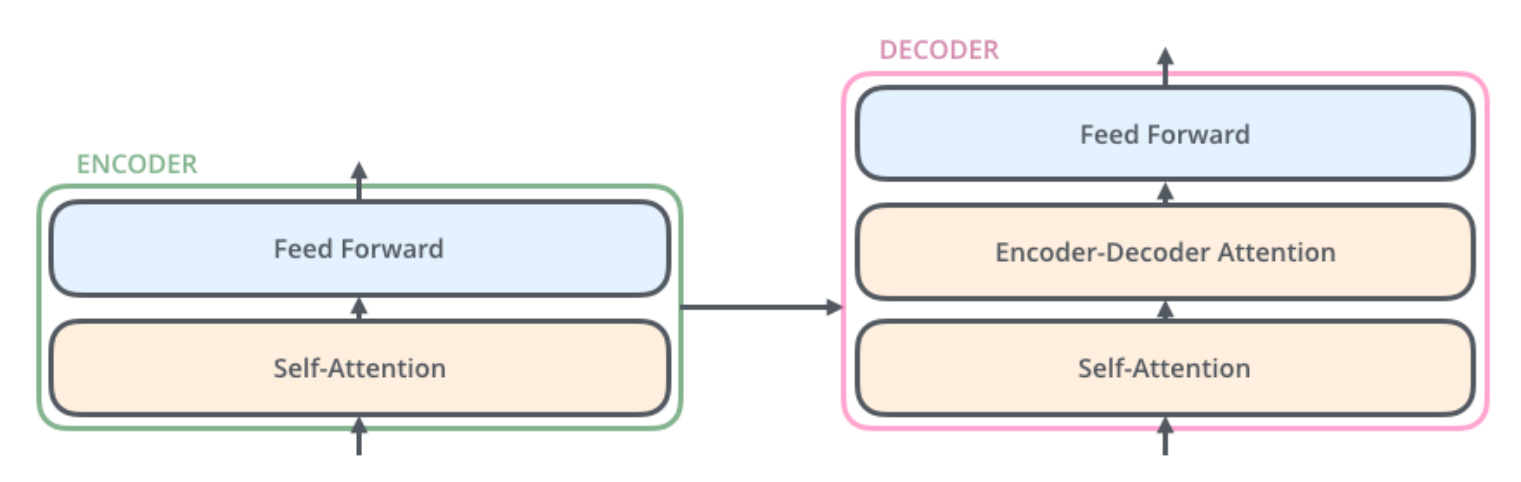


Figure 5: Again, all the decoders have the same structure. It has the same layers as the encoder, but with a layer in between that helps the decoder to focus on relevant parts of the input sequence.

Embeddings

Let's translate the sentence *Je suis étuiant* from French to English.

Figure 6: As always, the first step is to turn each word (or more specifically, each token) into its corresponding embedding. In the original paper, the embedding dimension $d_e = 512$.

First encoder

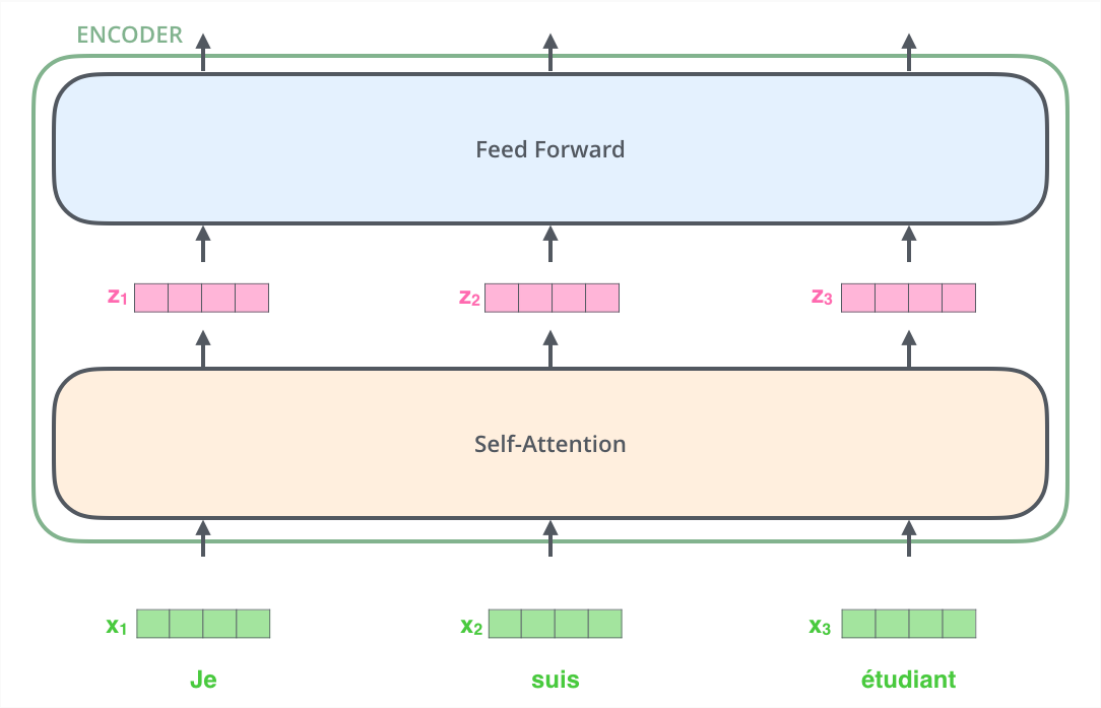


Figure 7: The first encoder receives a list of embeddings; i.e., a list of tensors with dimension d_e. **Note**: Each token follows its own path through the Transformer. This parallel structure is a key reason the model is highly parallelizable compared to RNNs.

.

Second and consecutive encoders

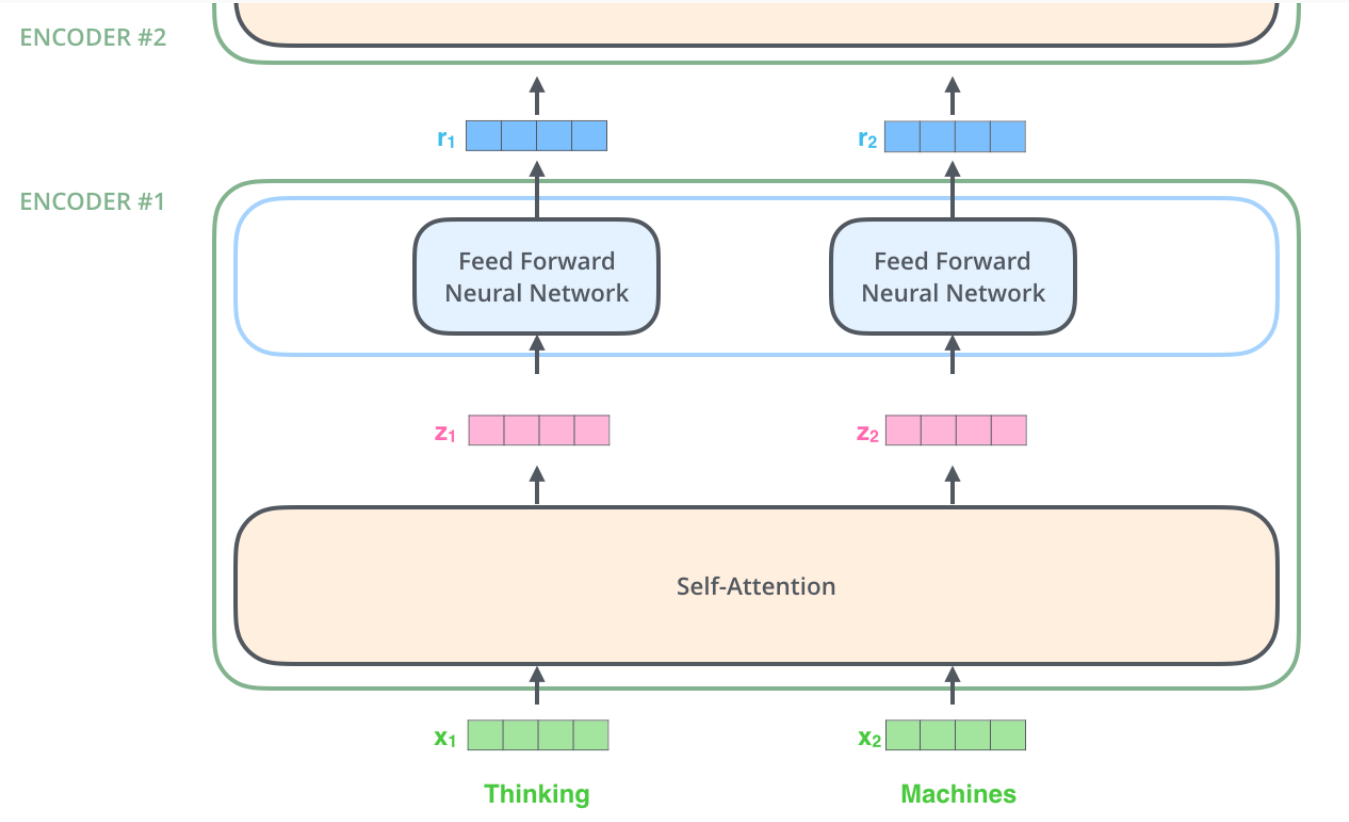


Figure 8: Each encoder layer (except the first) receives as input the output of the previous layer.

Self attention: Motivation I

We have mentioned self-attention several times, yet we still don't know what it is.

`The animal didn't cross the street because it was too tired`

Does `it` refers to `the animal` or `the street`?

Self attention: Motivation II

`The animal didn't cross the street because it was too crowded`

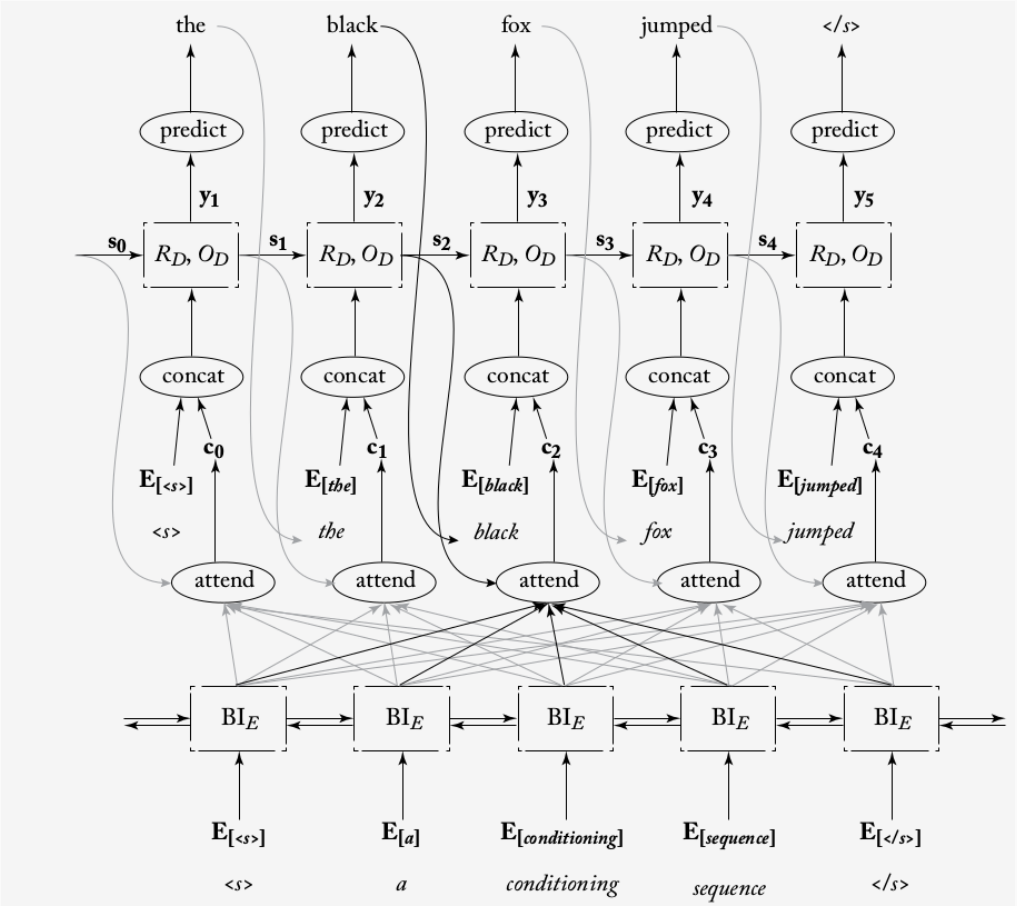
And now?

Self attention: Motivation III

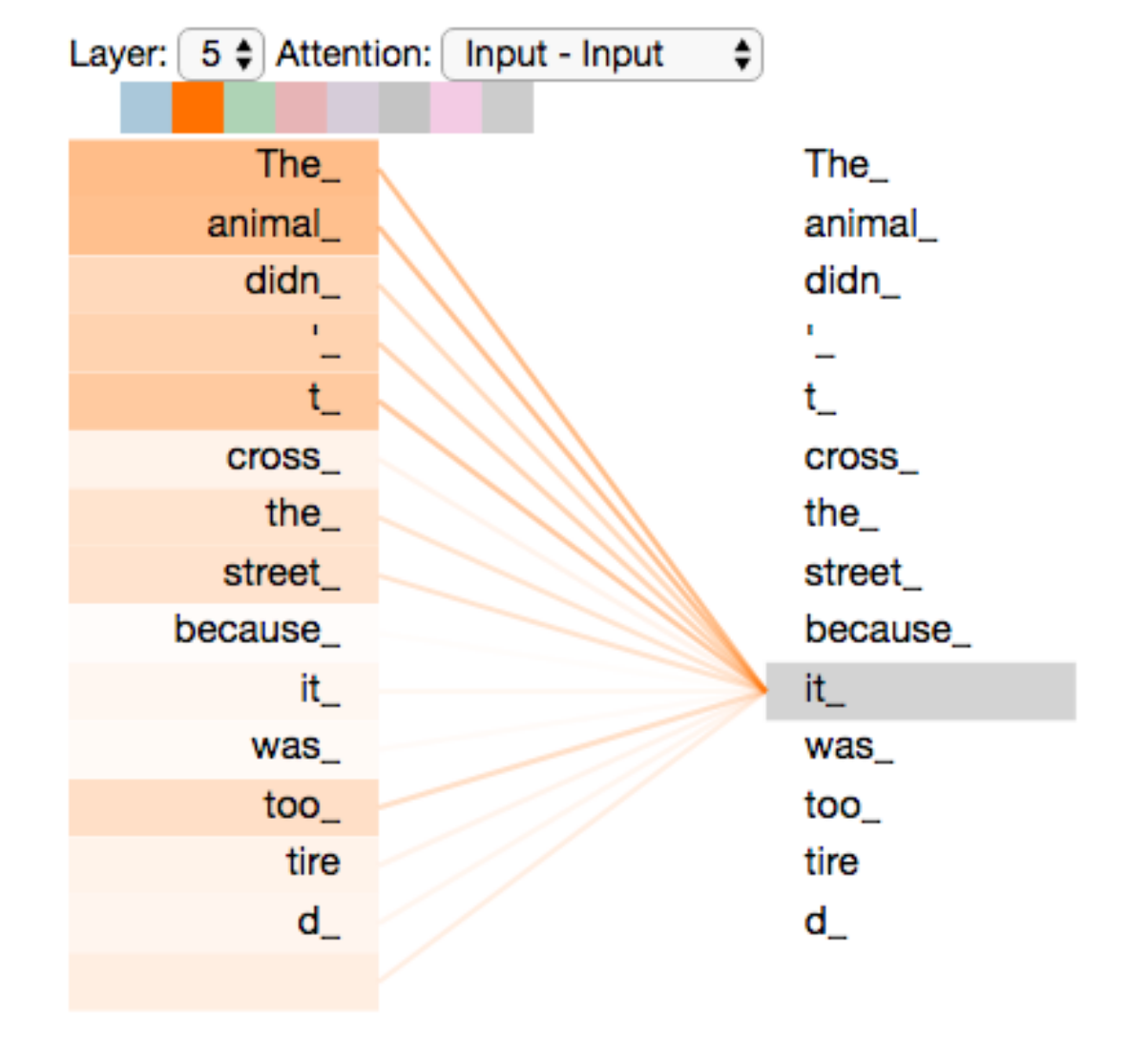
- Initial embeddings are *uncontextualized* — they don't account for polysemy or ambiguity (e.g., the word `it` in the previous example).
- Our goal is to produce *contextualized embeddings*, which reflect the meaning of a word based on its surrounding context.
- To do this, we leverage neighboring words so the model can learn which tokens provide relevant context for each position.

Self attention: Attention recap

- In the context of RNNs, we defined *attention* as a mechanism that allows the decoder to selectively focus on the encoder's hidden states based on its current state.
- In that setup, attention was used only to connect the encoder and decoder.
- Now we want to extend this mechanism to operate **within** the encoder and **within** the decoder — allowing each token to be contextualized using the entire sequence.
- This is called *self-attention*



Self attention: What we want to achieve



Self attention: framework

In the paper [Vaswani et al., 2017], attention is defined in terms of `queries`, `keys`, and `values`.

(Somewhat) high-level intuition of how it works:

- Suppose we have a query q, and a dictionary D containing keys $k_1, \ldots, k_n$ and associated values $v_1, \ldots, v_n$.
- Our goal is to retrieve the values v_i that are most relevant to the query q.
- To do this, we compute a similarity score α_i between q and each key k_i.
- We then normalize the scores so that $\sum_{i=1}^{n} \alpha_i = 1$ and $\alpha_i > 0$ for all i (typically using the softmax function).
- The final output is a weighted sum of the values:

$$\text{output} = \sum_{i=1}^{n} \alpha_i v_i$$

Self attention: framework

Two important ideas:

- Each query q, key k, and value v is a vector. $\rightarrow$ We can compute similarity between a query and each key using the dot product (or a variation of it).
- All queries, keys, and values are derived from the **same sentence** — this is what makes it **self-attention**.
- In the first encoder layer, each input embedding x_i is transformed as follows:
  - The query $q_i = W^Q x_i$
  - The key $k_i = W^K x_i$
  - The value $v_i = W^V x_i$
- In the second and subsequent encoder layers, the idea remains the same — but instead of transforming the original embeddings x_i, we now transform the outputs from the previous layer r_i into queries, keys, and values.

Self-attention: details (I)

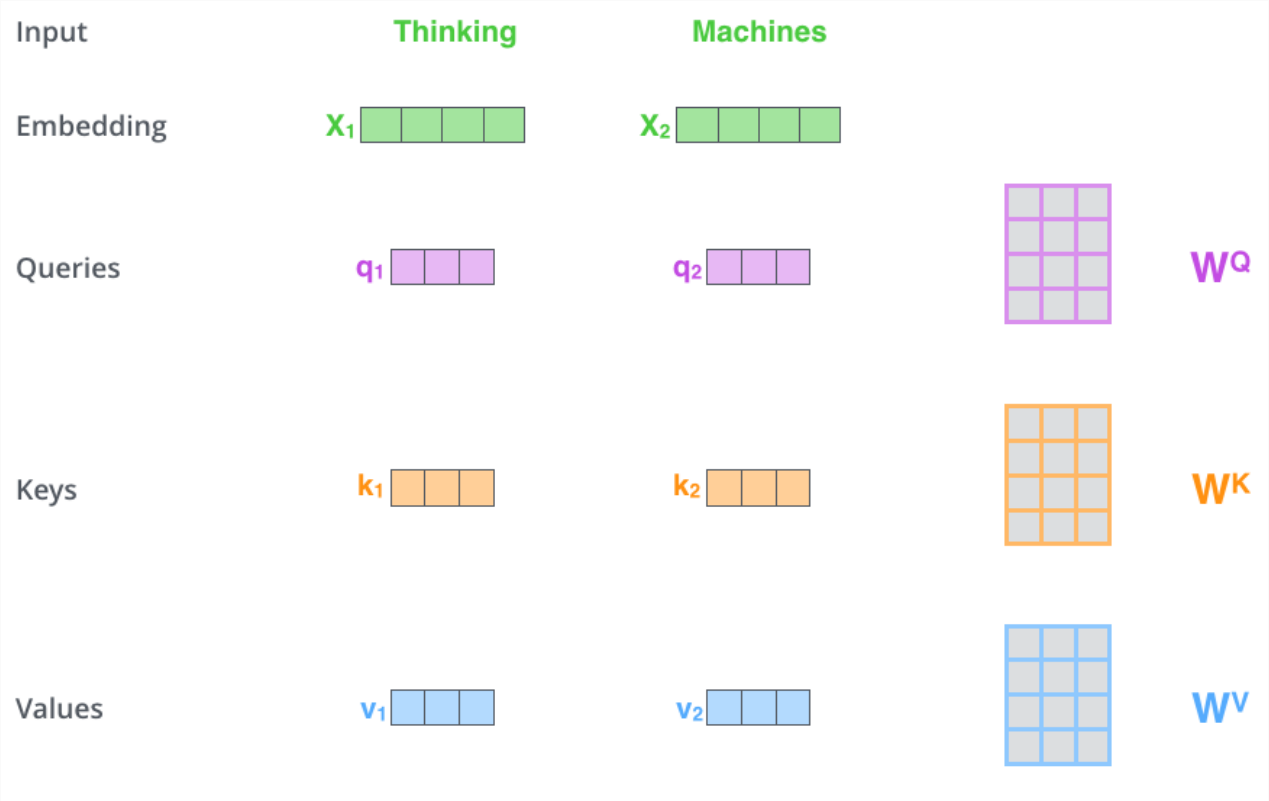


Figure 9: **First step**: Transform each embedding x_i (in the first encoder) or each output r_i from the previous layer (in subsequent encoders) into a query vector q_i, a key vector k_i, and a value vector v_i. **Note**: the dimensions of q, k, and v are typically smaller than the original vector. In the original paper, for example, $d_k = d_q = d_v = 64$ — but this is a design choice, not a strict requirement.

Self-attention: details (II)

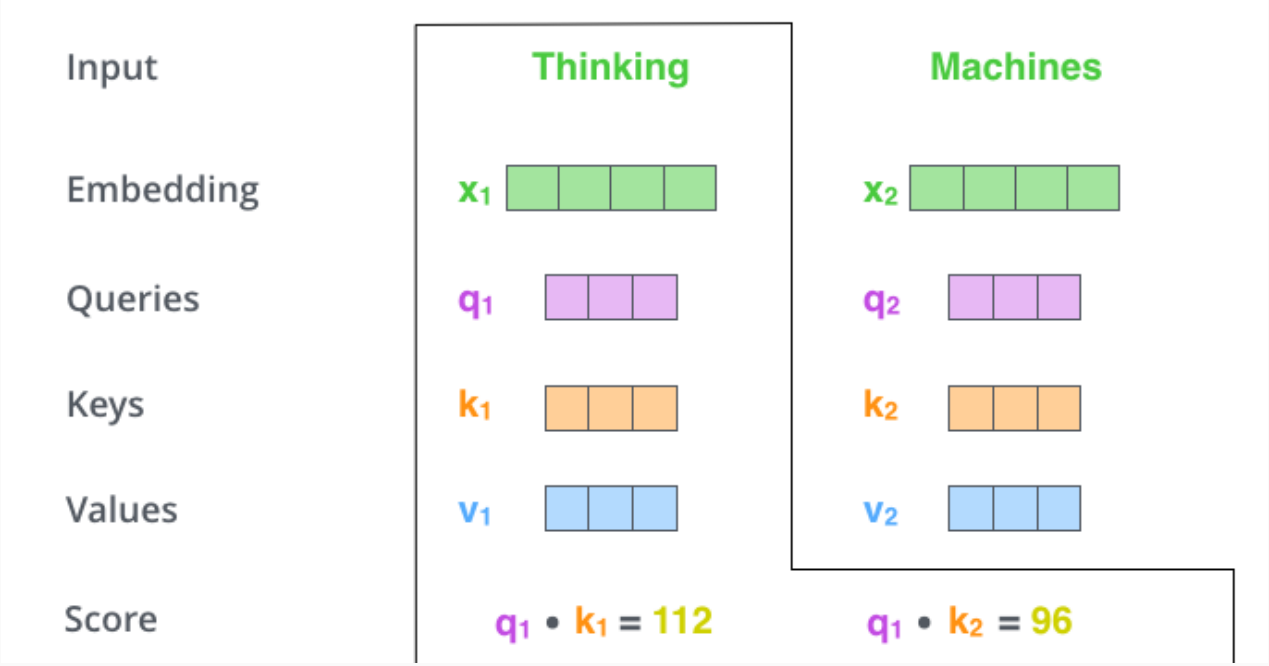


Figure 10: **Second step**: Calculate a score. Suppose we are computing self-attention for the word *Thinking*. We need to compute an score for each word in the sentence (i.e., for *Thinking* and *Machines*. This score tells us how much we should focus in each part of the sentence as we encode an specific word. This score is computed by taking the dot product between the query vector q and the corresponding key vector k.

Self-attention: details (III)

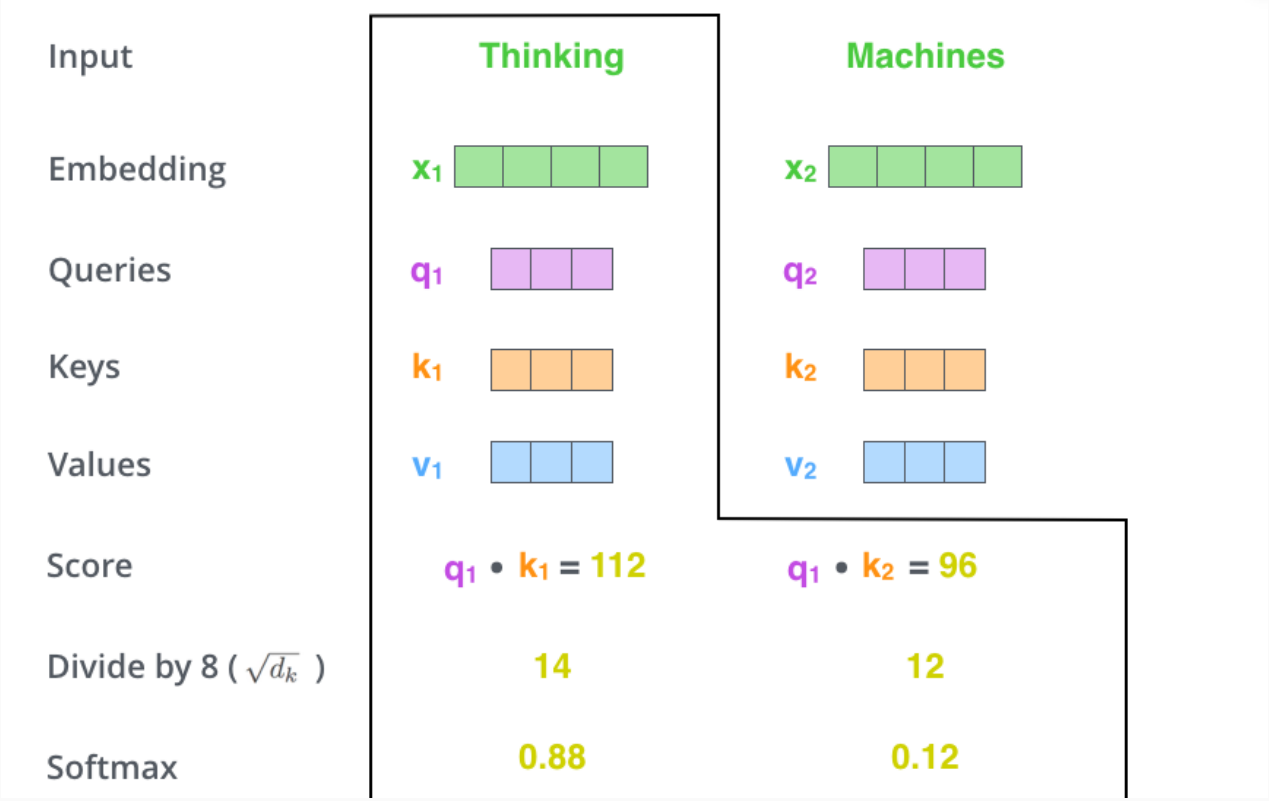


Figure 11: **Third step**: Divide the score by $\sqrt{d_k}$; this leads to more stable gradients. **Fourth step**: Apply the softmax function to the scaled scores to ensure they are all positive and sum to 1. The resulting normalized weights indicate how much attention should be paid to each word in the sequence. **Note**: this values are called *attention weights*.

Self-attention: details (IV)

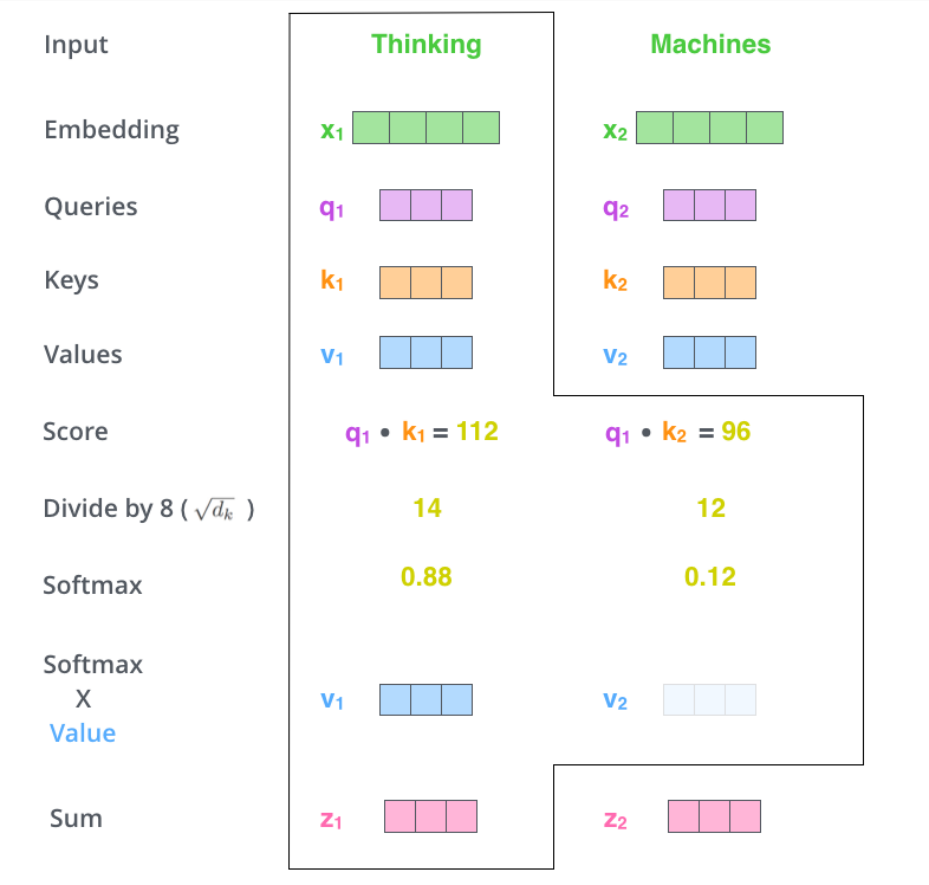


Figure 12: **Fifth step**: Multiply each value vector by its corresponding attention weight. This emphasizes the values of the words we want to focus on and suppresses less relevant ones (by scaling them down with small weights, like 0.001). **Sixth step**: Sum the weighted value vectors. This gives the output of the self-attention layer at this position — in this example, for the word `Thinking`.

Self-attention: details (V)

- That completes the self-attention computation!
- The resulting vector is then passed through a feed-forward network and forwarded to the next encoder layer.
- However, to make this process efficient, we perform all these operations using matrix computations. Let's see how that works.

Self-attention: matrix calculation

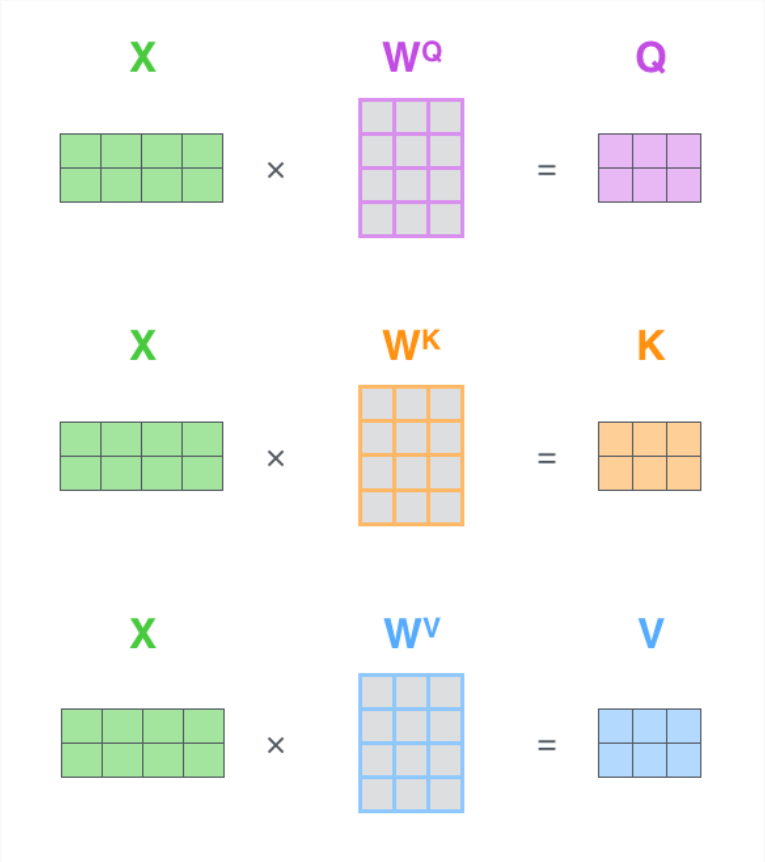


Figure 13: **First step**: As before, the first step is to generate the queries, keys, and values. We do this by stacking all input embeddings into a matrix X, then multiplying it with the weight matrices W^Q, W^K, and W^V to obtain the matrices Q, K, and V. Note that, originally, each embedding x_i has dimension $d_e = 512$ (represented as four boxes), and after projection, each q_i, k_i, and v_i has dimension 64 (three boxes).

Self-attention: matrix calculation

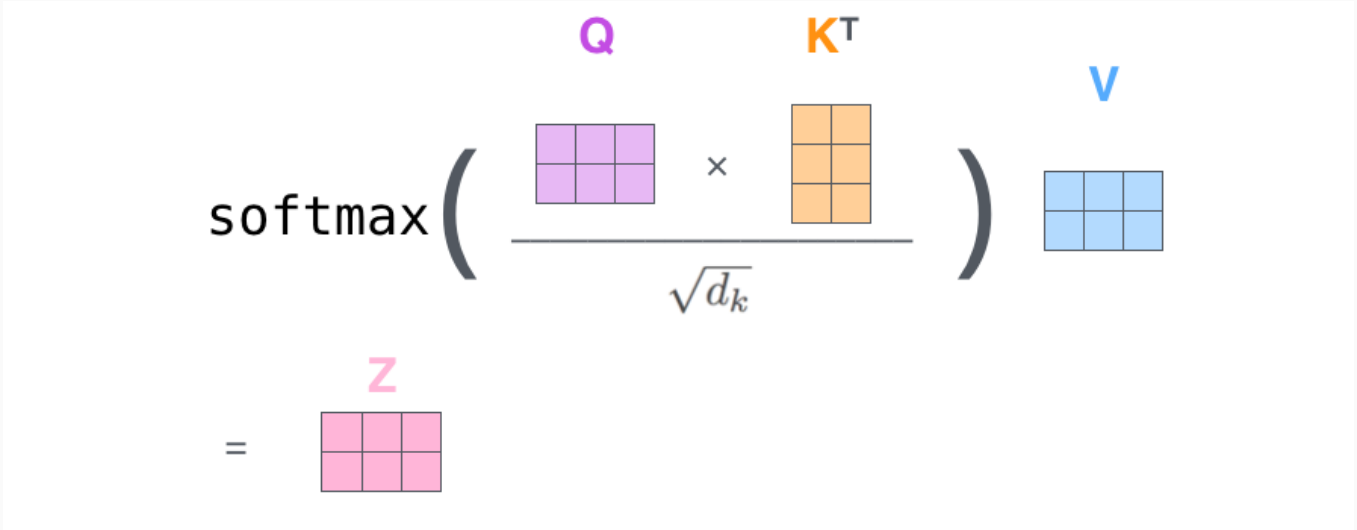


Figure 14: **Steps 2 through 6**: Since we are dealing with matrices, we can condense everything into one formula to compute the outputs of the self-attention layer.

Multi-head attention

- The original paper further refined the self-attention layer by introducing a mechanism called *multi-head attention*.
- The idea is to run several self-attention mechanisms in parallel and then combine their outputs (more on that shortly).
- This provides two key benefits:
  - It expands the model's ability to attend to different positions. If one token dominates attention in a single head, other heads can focus elsewhere.
  - It enables multiple "representation subspaces." Each head uses its own set of W^Q, W^K, and W^V matrices, projecting the inputs into different subspaces.
- In the original paper, they used $H = 8$ attention heads. Each head projects the inputs into 64-dimensional spaces ($512/8 = 64$), which keeps the total dimensionality and computational cost comparable to using a single 512-dimensional attention head.

Multi-head attention: details (I)

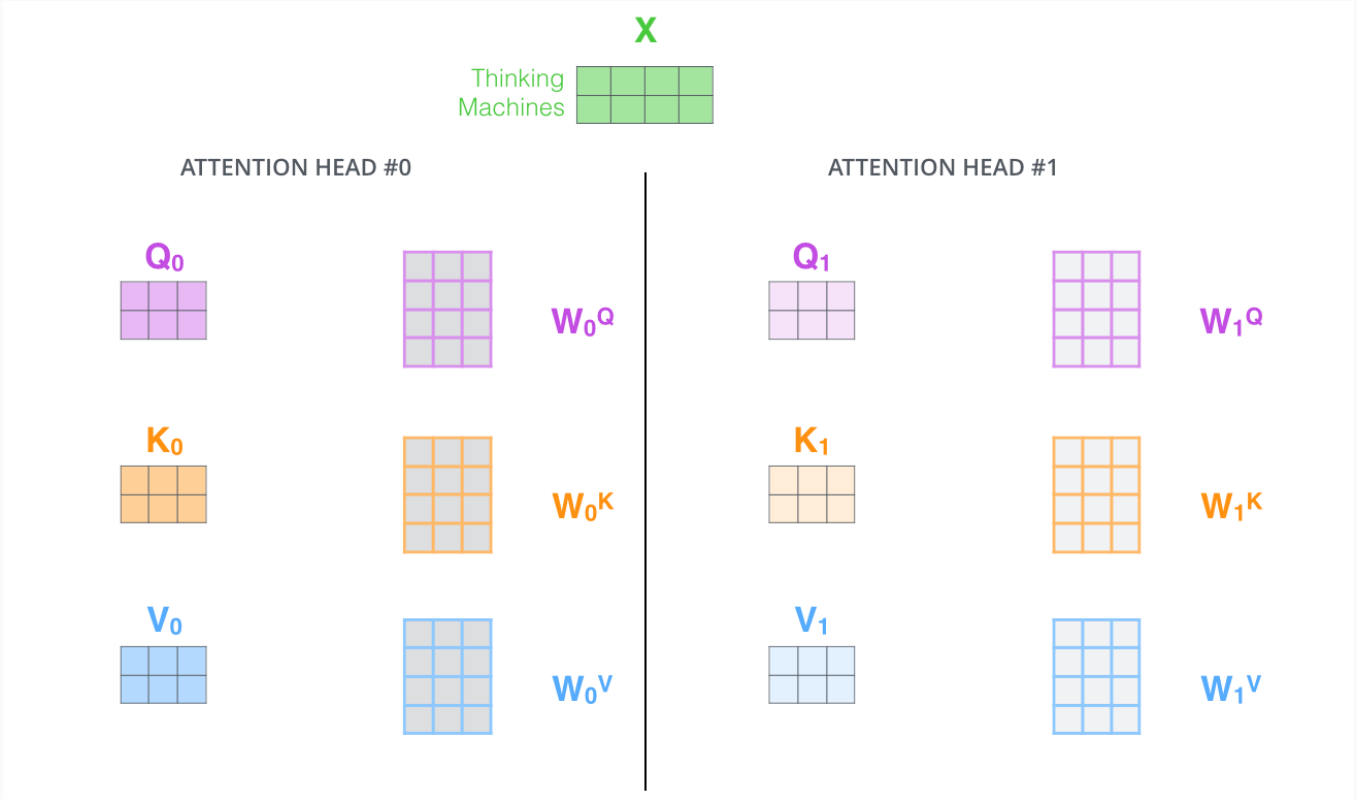


Figure 15: For each attention head, we use a separate set of weight matrices W^Q, W^K, and W^V, which results in different Q, K, and V matrices for each head. As we did before, we multiply X by W_i^Q, W_i^K, and W_i^V to obtain Q_i, K_i and V_i, where i is the head index.

Multi-head attention: details (II)

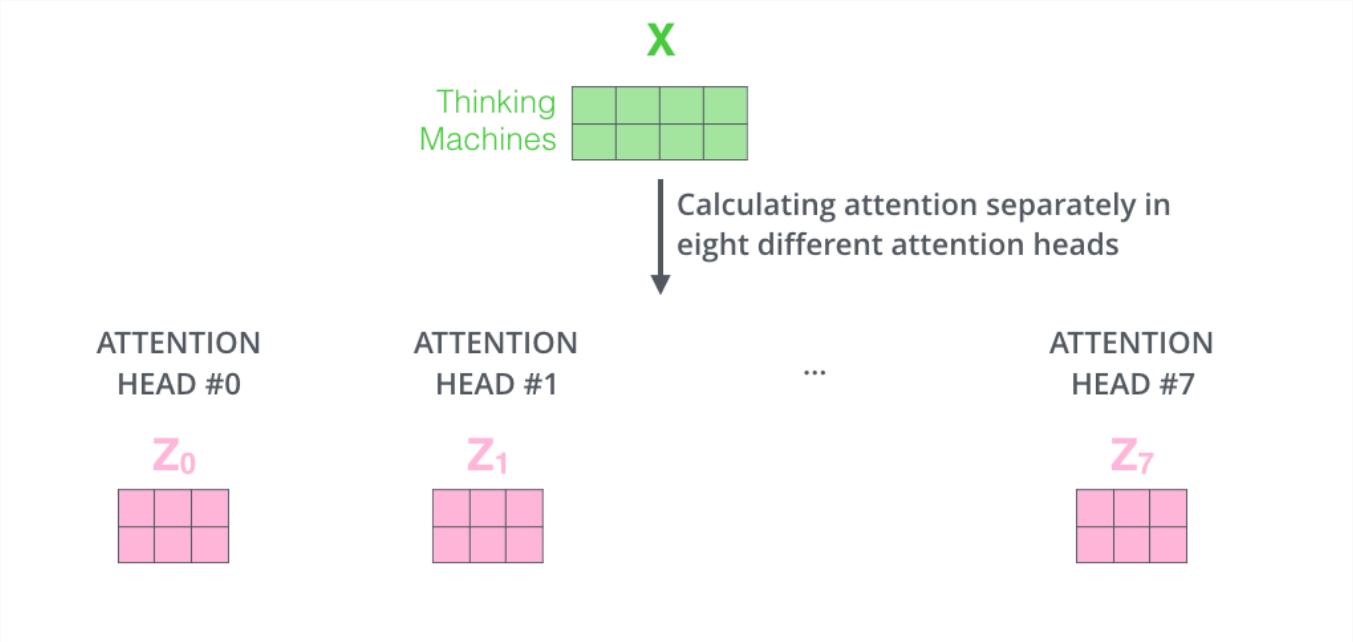


Figure 16: If we follow the procedure previously outlined but with $H = 8$ heads, we end up with eight different Z matrices.

Multi-head attention: details (III)

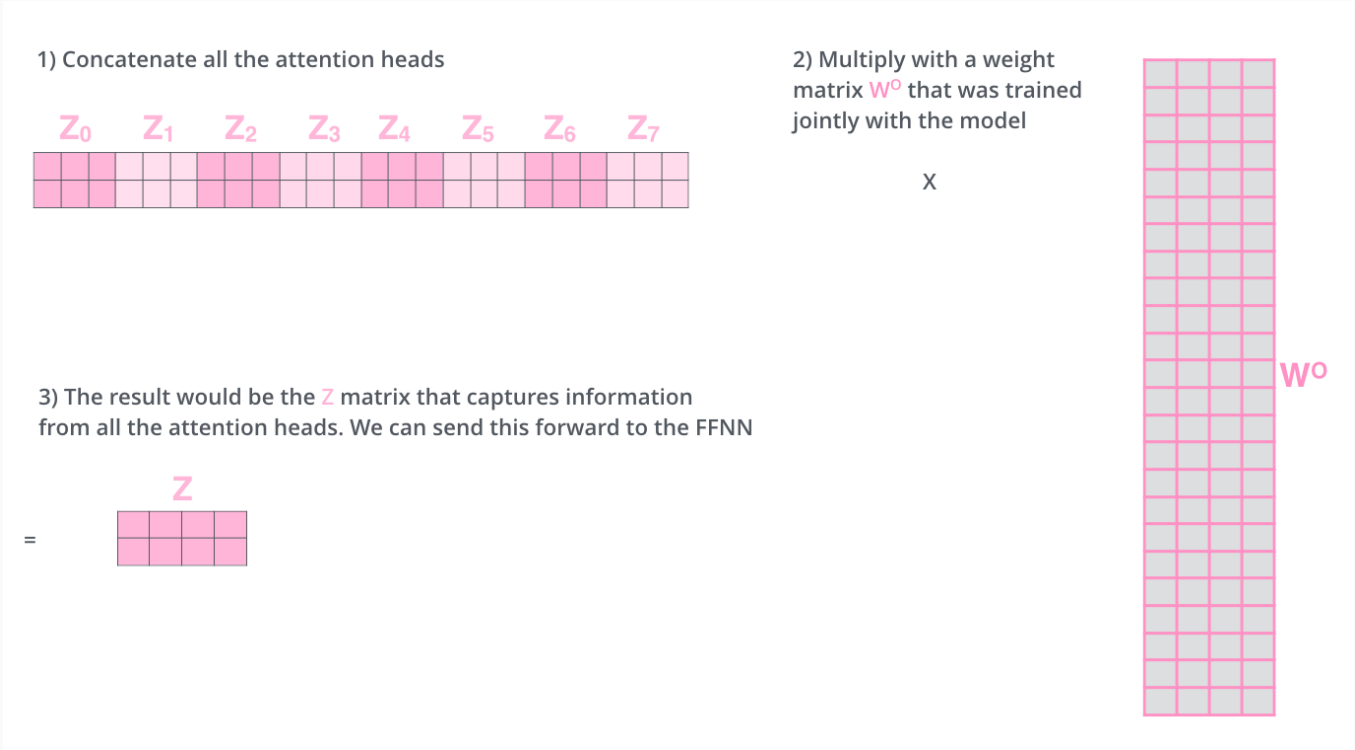
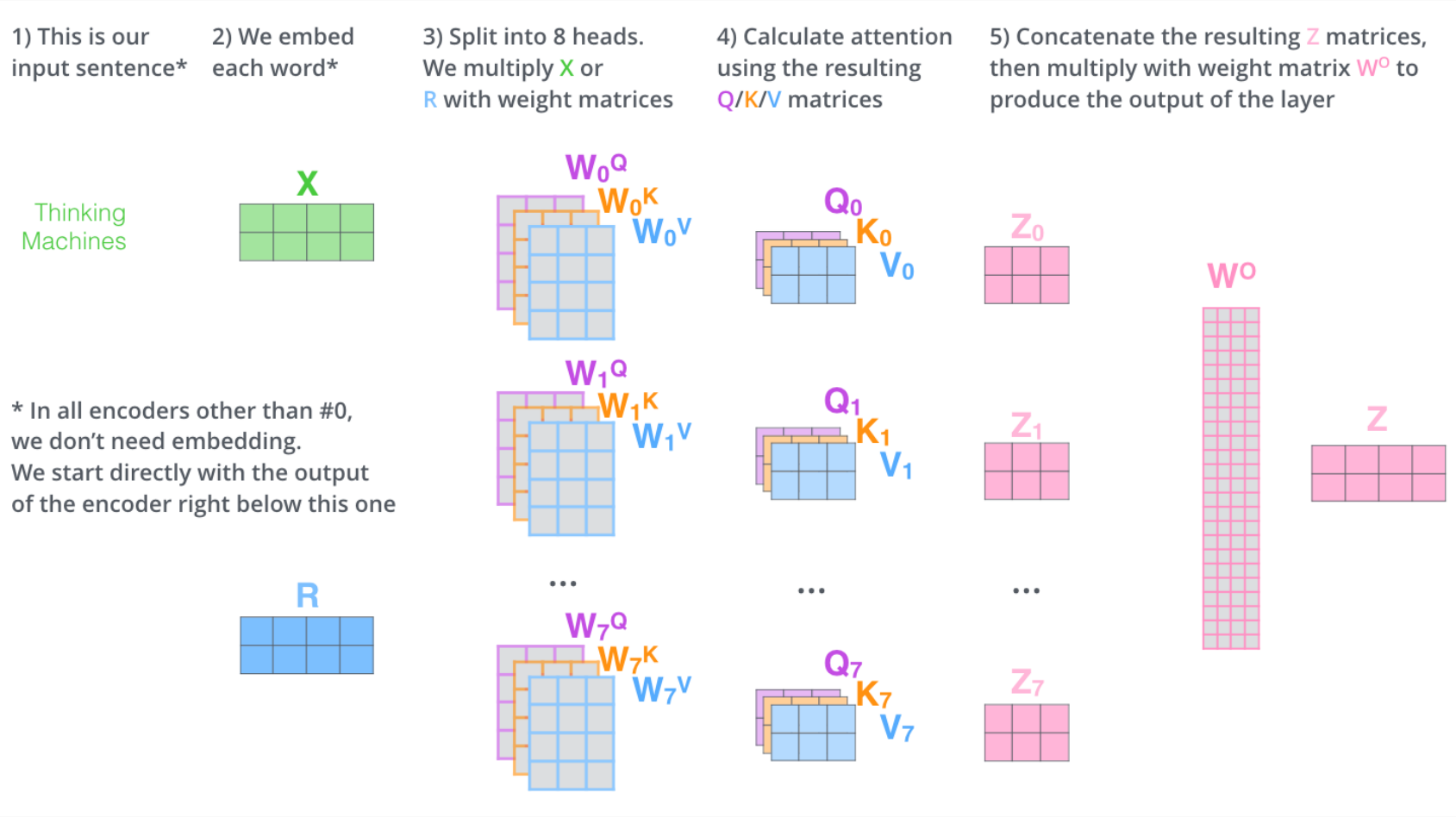


Figure 17: The feed-forward layer expects a single matrix — with one vector per word — not H separate matrices. To resolve this, we concatenate the H attention head outputs and project the result using an additional weight matrix W^O.

Multi-head attention: all together



Multi-head attention: result

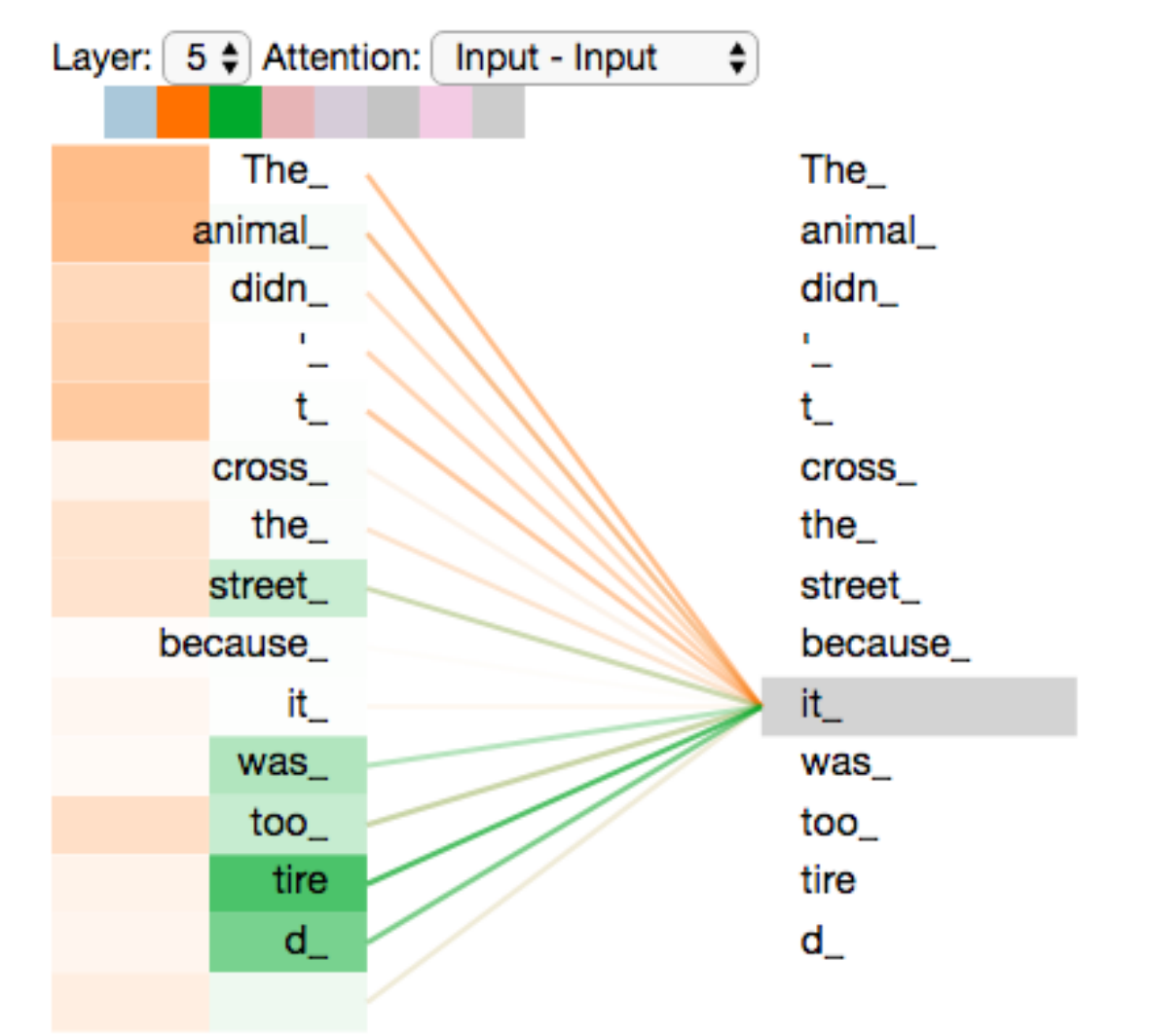


Figure 18: Different attention heads can focus on different parts of the sentence, capturing diverse patterns and relationships.

Positional Encoding

- The first encoder processes a list of embeddings — and this can be done in parallel.
- However, that list has no inherent order; it's essentially a bag of words.
- To handle sequential information, we need a way to represent word order.
- We achieve this by adding a position vector to each input embedding. These vectors follow a specific, learnable pattern that helps the model infer each word's position and the relative distances between words.

Positional encoding

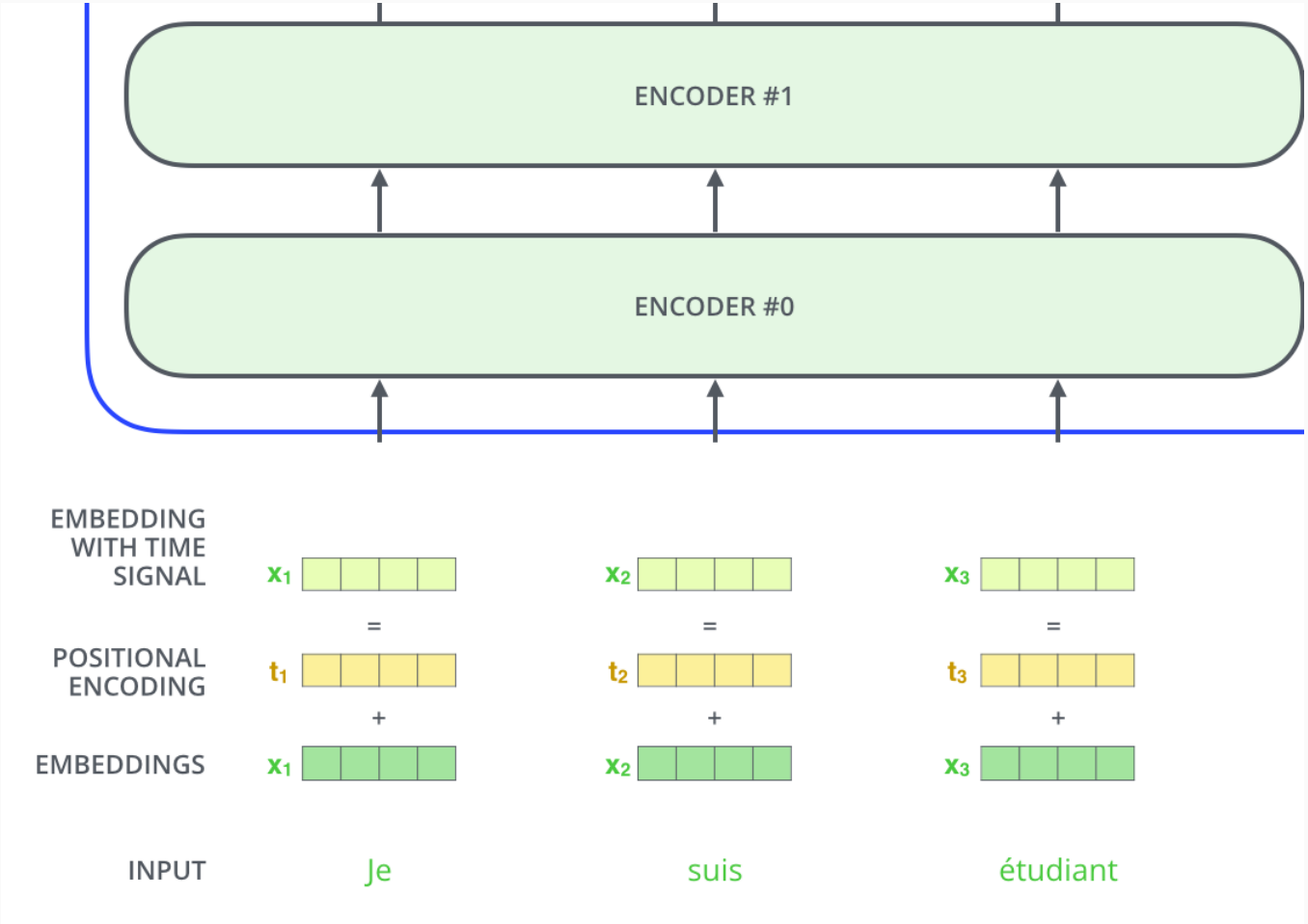


Figure 19: Addition of positional encoding vectors.

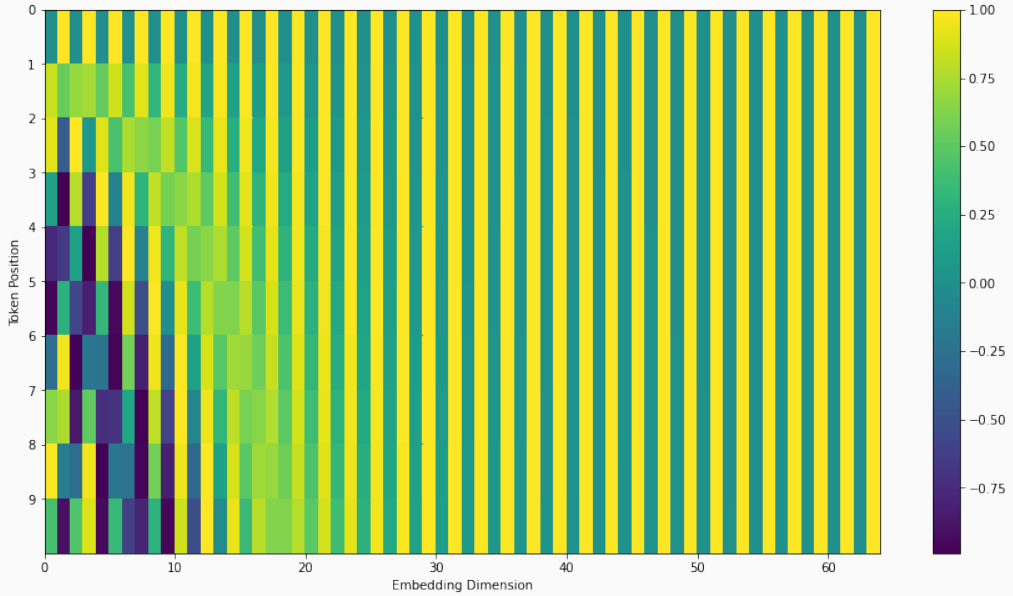
Positional Encoding

- While it's possible to learn positional encodings, the original paper used fixed ones based on sine and cosine functions.
- The formula they used is:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

- In other words, the position of each token is encoded using two interleaved signals: sine for even dimensions and cosine for odd ones.



Positional encoding: example

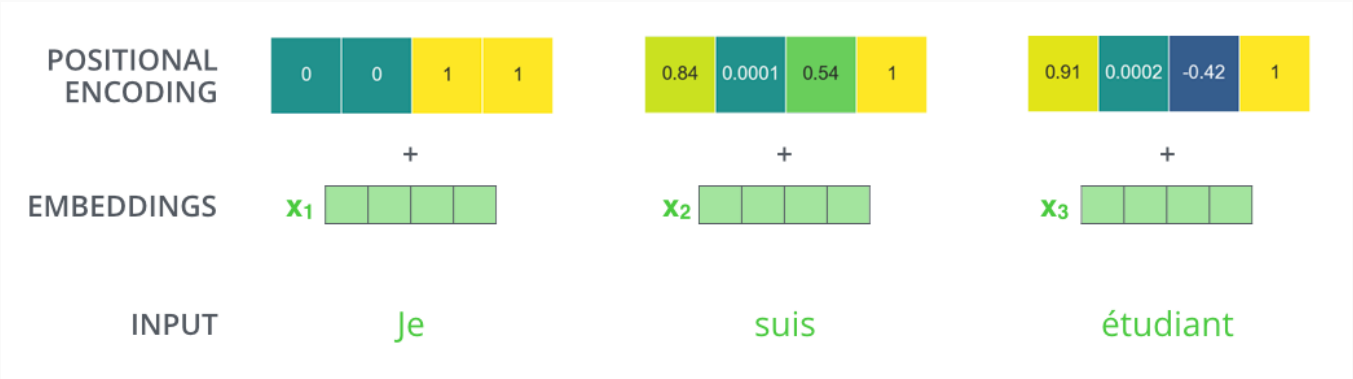


Figure 20: A real example of positional encoding with a toy embedding size of 4 and a function similar (but not the same) to the one presented in the paper.

Some architectural details

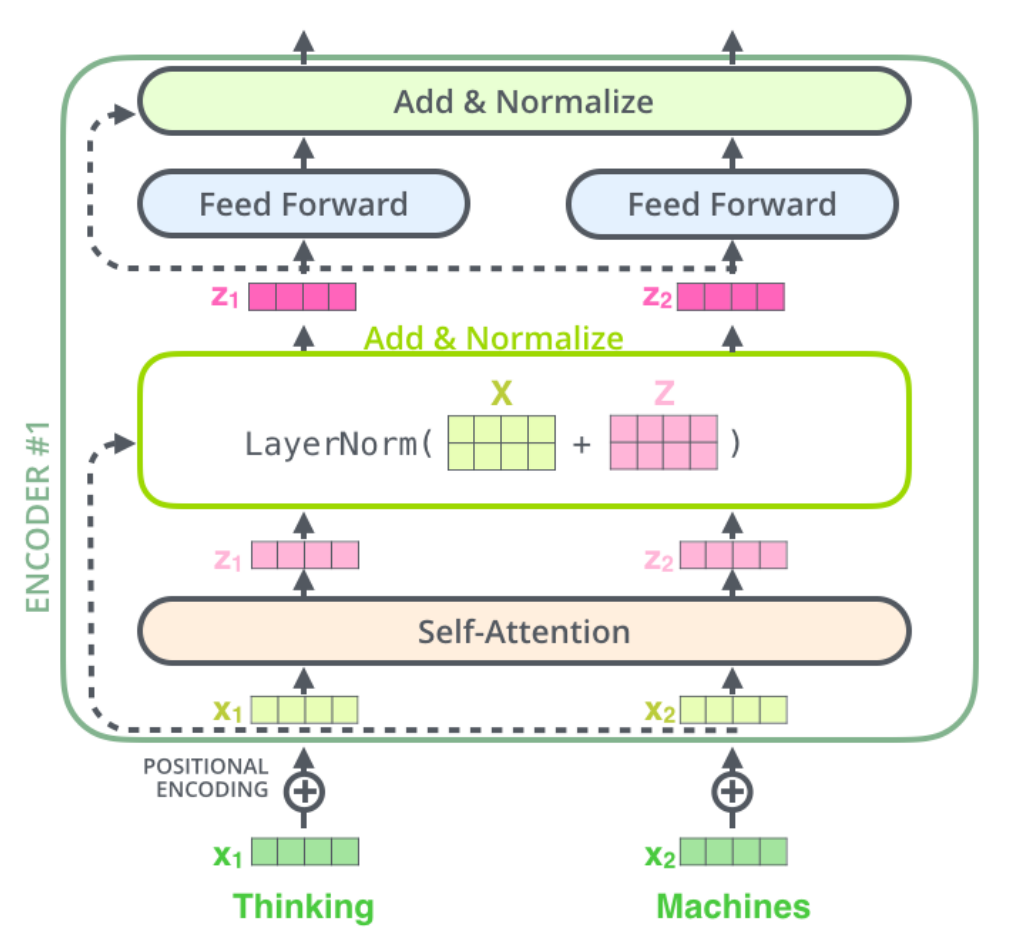


Figure 21: Each sublayer (self-attention, ffn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.

Some architectural details

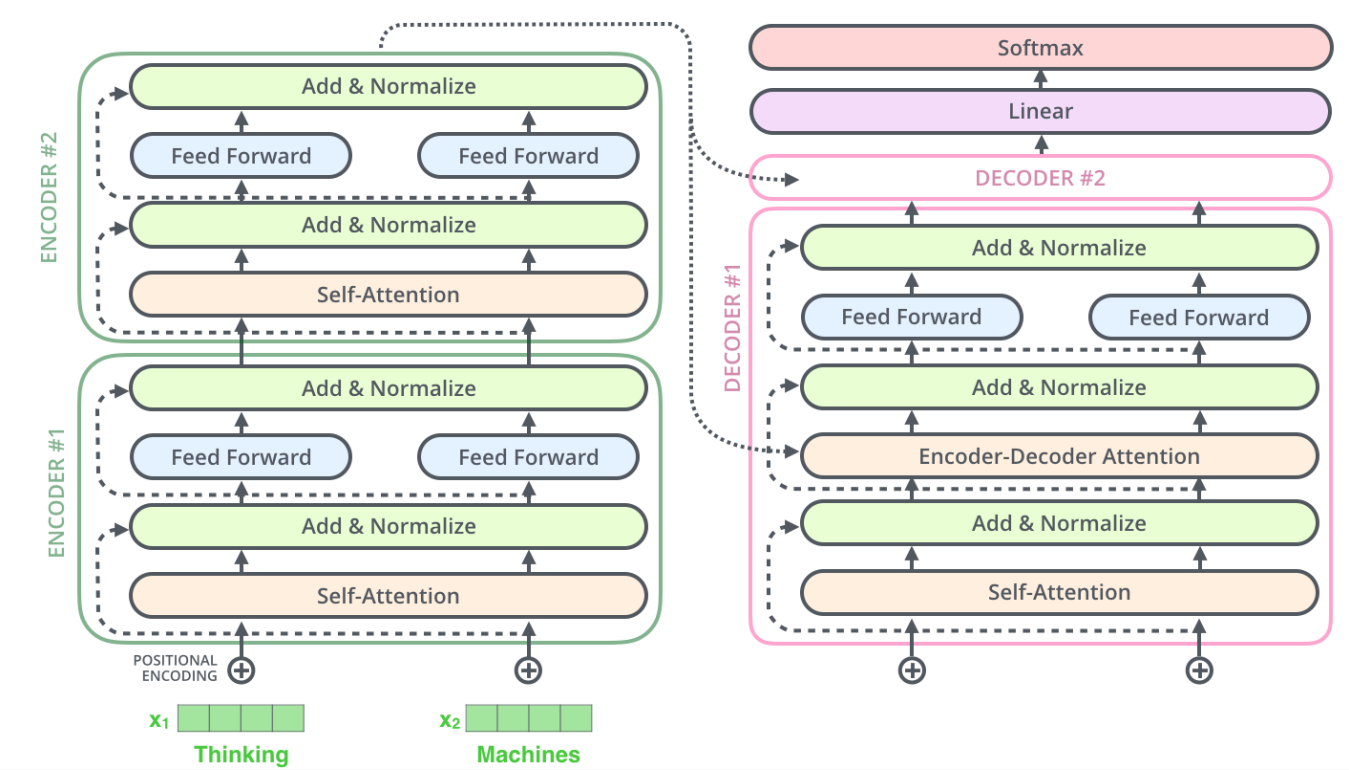


Figure 22: This goes for the sub-layers of the decoder as well.

The decoder side (I)

Let's focus on inference for the moment.

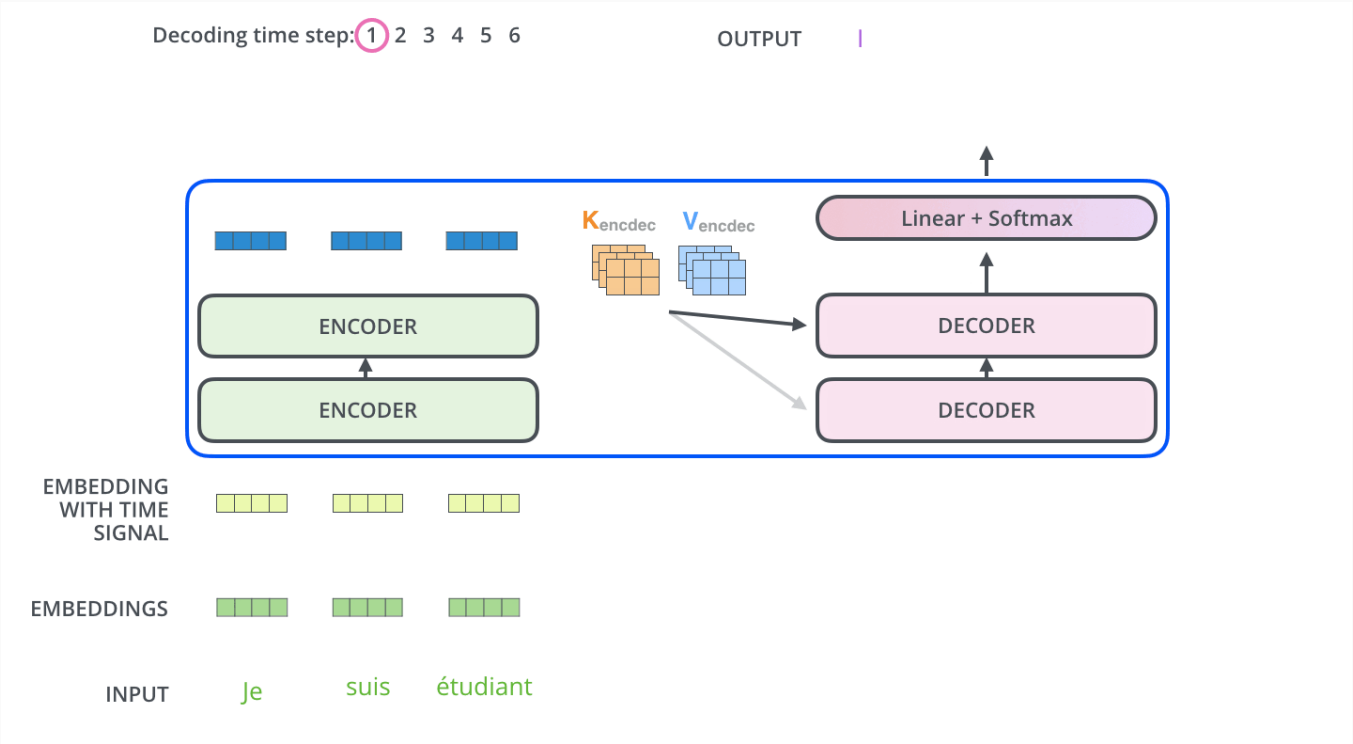


Figure 23: The encoder processes the input sequence, and the output of the top encoder is transformed into key (K) and value (V) vectors. These are used by each decoder in the encoder-decoder attention layer to focus on relevant parts of the input sequence. The queries (Q) come from the output of the previous decoder layer. Decoding begins with the special start-of-sequence token `<bos>`. Animated version here

The decoder side (II)

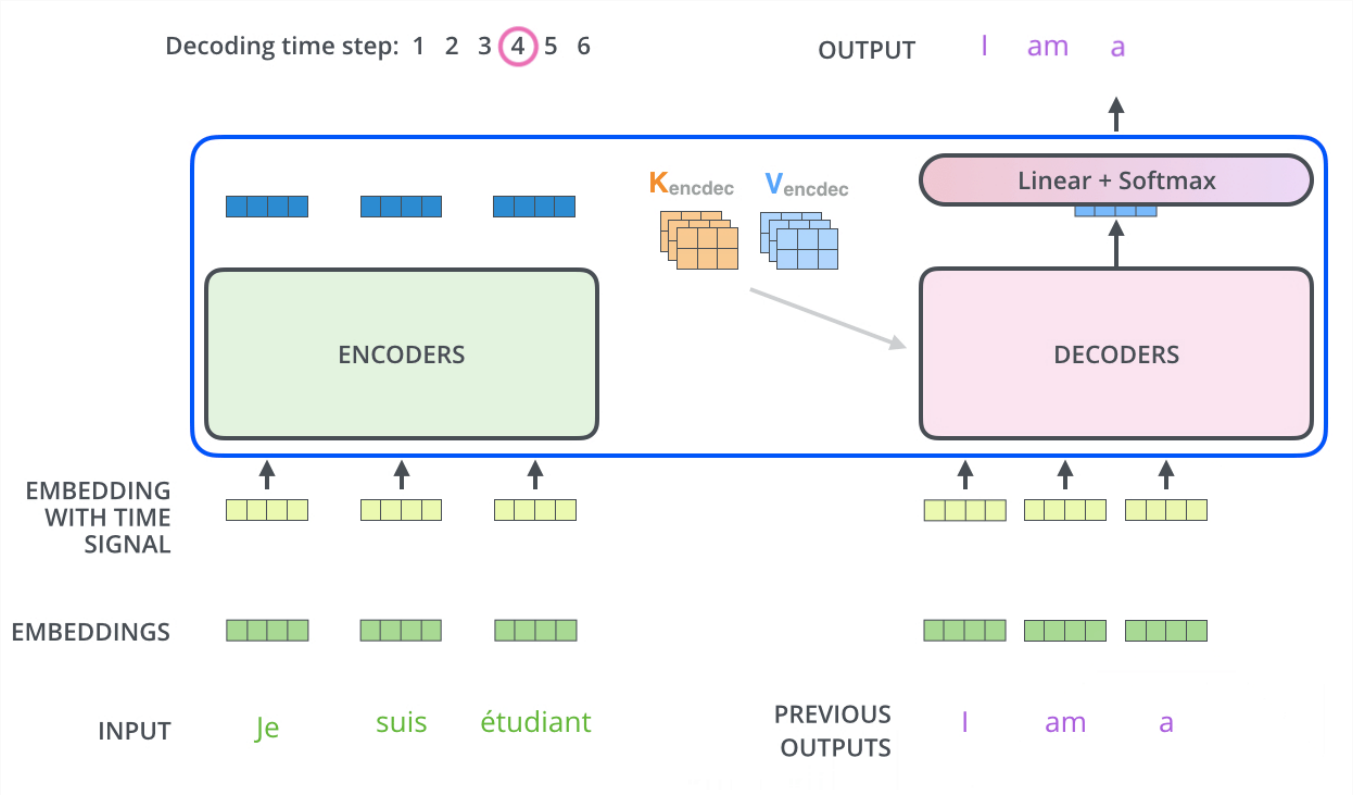


Figure 24: The decoding continues until a special end-of-sequence `<eos>` symbol is reached. The output at each time step is fed into the bottom decoder layer at the next time step, and the decoders propagate their results upward through the stack, just like the encoders did. Animated version here

The decoder side (III): masked self-attention

- At each decoding step, we feed the decoder with all previously generated tokens. In theory, all attention scores and contextualized embeddings must be recomputed.
- For example, when generating the word `"am"`, only the earlier tokens are available — and that's expected. But suppose we've now generated `"I am a"` and feed the full sequence through the decoder.
- At this point, the word `"am"` should not have access to `"a"`, because `"a"` was generated later. Allowing this would be like letting the model peek into the future.
- To prevent this, the decoder uses *masked self-attention*: it is only allowed to attend to earlier positions in the sequence. This is implemented by masking future positions (i.e., setting them to $-\infty$) before applying the softmax.

The decoder side (IV): masked self-attention

- At inference time, attention values from previous steps are cached (known as *KV caching*), so we don't need to recompute everything or worry much about masked self-attention at that moment. (See `https://medium.com/@joaolages/kv-caching-explained-276520203249` for more details.)
- However, masked self-attention is crucial during training: the decoder is fed the entire target sequence at once (via *teacher forcing*), and each position should only attend to earlier tokens.

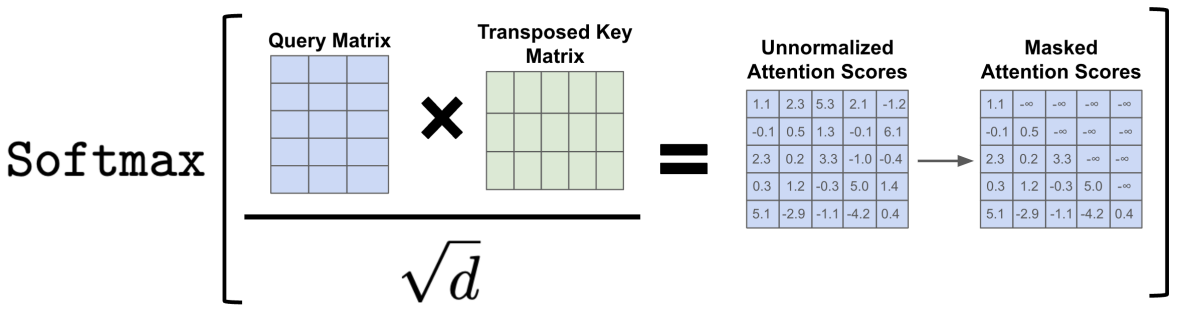


Figure 25: Masked self-attention. Source: https://cameronrwolfe.substack.com/p/decoder-only-transformers-the-workhorse.

The decoder side (V): The Final Linear and Softmax Layer

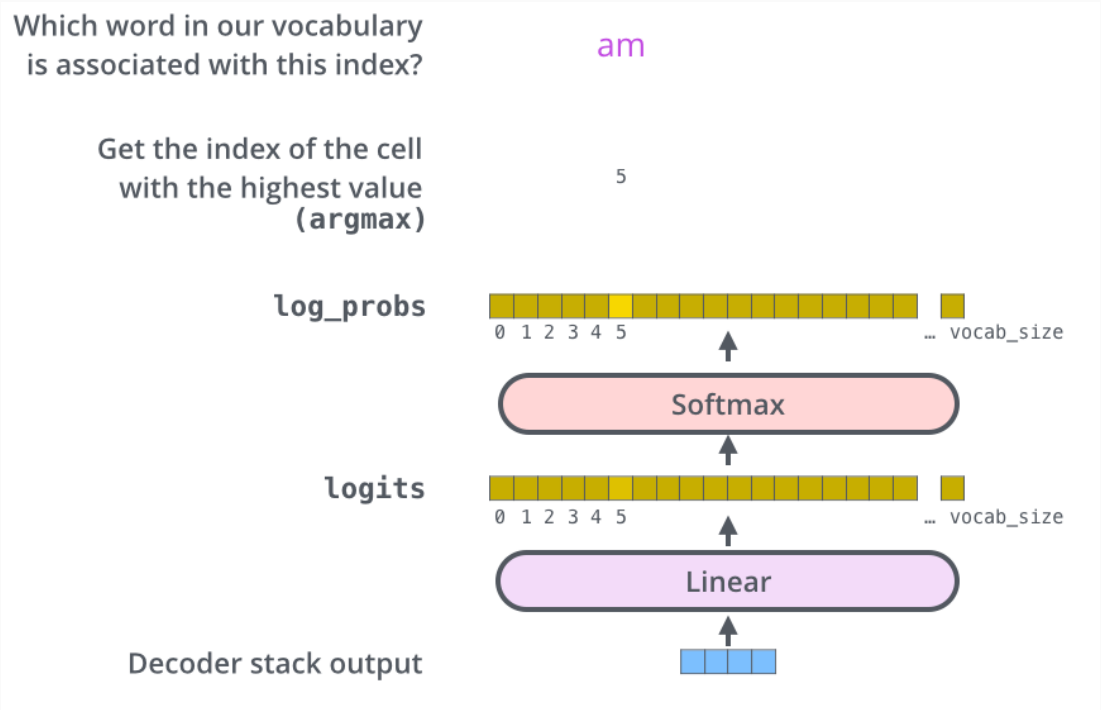


Figure 26: The decoder stack outputs a vector of floats that needs to be mapped to a word. To do this, we apply a Linear layer followed by a Softmax layer. The Linear layer projects the decoder output into a much larger vector — the *logits vector* — whose size equals the target vocabulary size. The Softmax layer then converts these scores into probabilities. The cell with the highest probability is typically selected, and the corresponding word is produced as the output for this time step. **Note:** Instead of selecting the most probable word, it is also possible to sample from the distribution.

Training

- So far, we have mainly focused on inference time — i.e., a forward pass through a trained Transformer.
- During training, the model goes through the same forward pass. However, since we have labeled data, we can compare the model's predicted output with the correct (ground-truth) output.
- For simplicity, let's assume a target vocabulary of just six words. We represent each target word using one-hot encoding.

Output Vocabulary

WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"

0.0	1.0	0.0	0.0	0.0	0.0

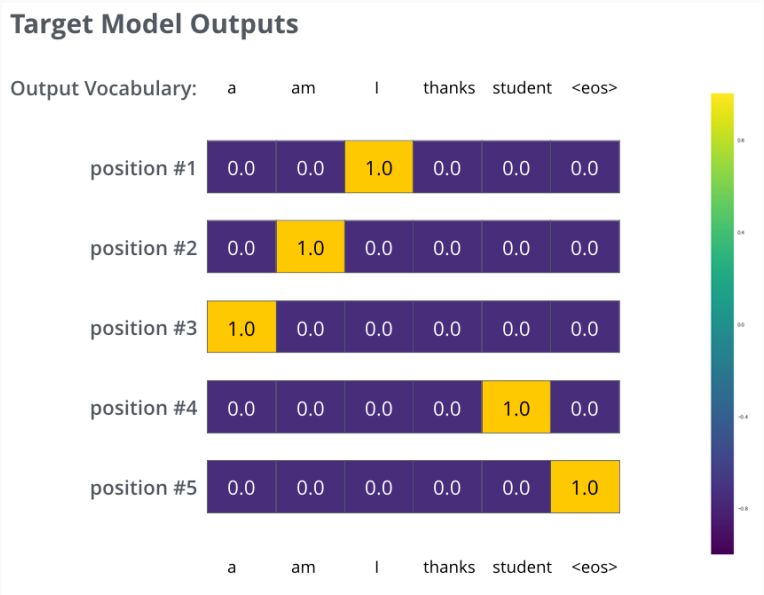
Training (II)

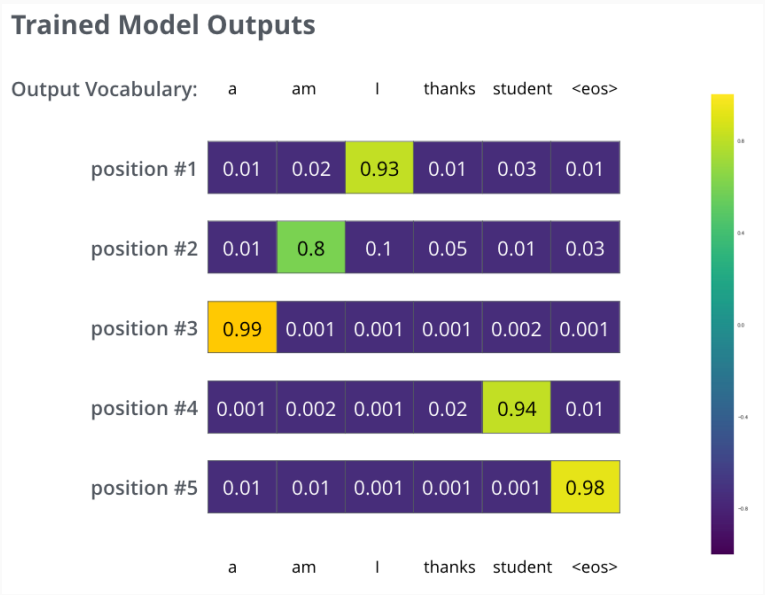
- Suppose we are at the very beginning of training, and we are training on a simple example: translating `merci` into `thanks`.
- Our goal is for the model to produce a probability distribution with a high value at the position corresponding to the word `thanks`.
- However, since the model is untrained, this is unlikely to happen early on.
- Both the predicted output and the target are probability distributions, so we can use the cross-entropy loss to measure the difference between them and update the model accordingly.

Training (IV)

- In practice, we usually train on full sentences, not just single words.
- For example, suppose the input is “je suis étudiant” and the expected output is “i am a student”.
- In this case, we want the model to generate a sequence of probability distributions such that:
  - Each distribution is a vector of size `vocab_size`.
  - The first distribution assigns the highest probability to the word “i”.
  - The second distribution peaks at the word “am”.
  - The third and fourth distributions target “a” and “student”, respectively.
  - The final distribution corresponds to the special token `<eos>`.

Training (V)


Target Model Outputs
Output Vocabulary: a am I thanks student <eos>
position #1 0.0 0.0 1.0 0.0 0.0 0.0
position #2 0.0 1.0 0.0 0.0 0.0 0.0
position #3 1.0 0.0 0.0 0.0 0.0 0.0
position #4 0.0 0.0 0.0 0.0 1.0 0.0
position #5 0.0 0.0 0.0 0.0 0.0 1.0
a am I thanks student <eos>


Trained Model Outputs
Output Vocabulary: a am I thanks student <eos>
position #1 0.01 0.02 0.93 0.01 0.03 0.01
position #2 0.01 0.8 0.1 0.05 0.01 0.03
position #3 0.99 0.001 0.001 0.001 0.002 0.001
position #4 0.001 0.002 0.001 0.02 0.94 0.01
position #5 0.01 0.01 0.001 0.001 0.001 0.98
a am I thanks student <eos>

Finally...

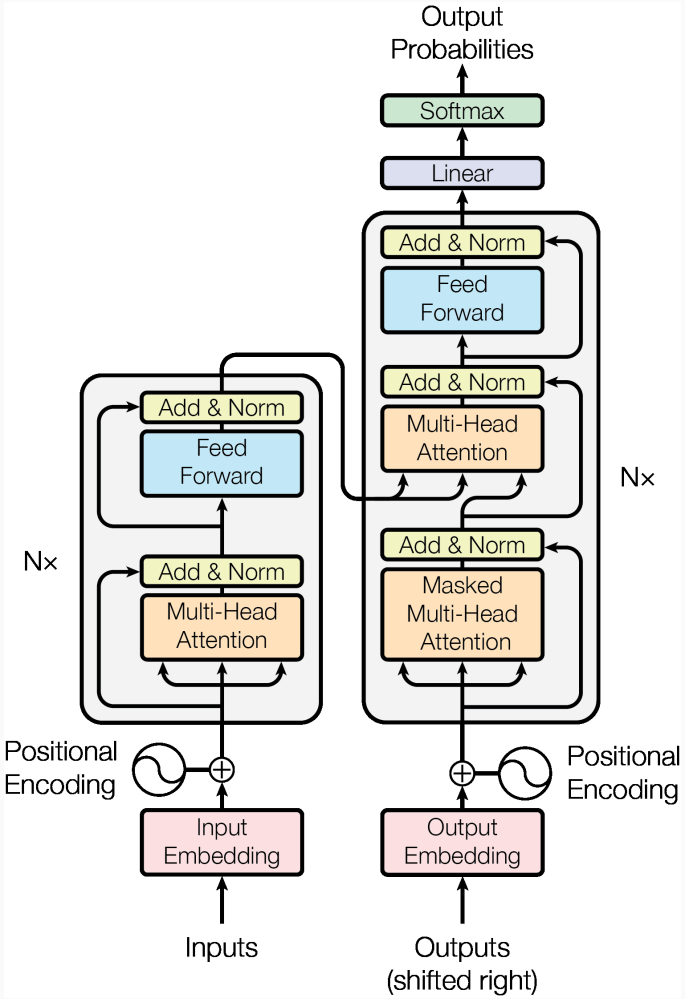


Figure 27: Transformer architecture, as depicted in the original paper [Vaswani et al., 2017]

Finally?

- Today, we explored the original Transformer architecture, which consists of an encoder and a decoder.
- There are also variations of this architecture, such as encoder-only models (e.g., BERT) and decoder-only models (e.g., GPT).
- We'll discuss these alternative architectures in the next lesson.

References i

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). **Attention is all you need.** *Advances in neural information processing systems*, 30.